



Solving Puzzles with SAT Solvers

Stop writing algorithms. Start stating the rules.

Meet Z3

What on Earth is Z3?

- A **SAT solver** (and more!) from Microsoft Research
- Solves boolean satisfiability problems: can we make these constraints true?
- You describe **what** must be true
- Z3 works out **how** — or proves it's impossible
- *(Bonus: Z3 also handles integers, reals, arrays, etc. — it's a full SMT solver! You will learn this later.)

You, the wizard

“Exactly one room hides the Lady.
Exactly one sign on the doors is
telling the truth.”

Z3, the genie

“Then she is behind door one.” ✓
[R1 = True, R2 = False, R3 = False,
...]

Declarative, not procedural

Normal programming

- You write the *steps*
- **You** are the algorithm designer
- Loops, conditionals, backtracking. . .
- Bug in your search $\Rightarrow$ wrong answer

Z3 programming

- You write the *constraints*
- **Z3** is the algorithm designer
- You just state the rules of the game
- Rules right $\Rightarrow$ answer right

You bring the rules.
Z3 brings the answer.

Under the Hood: SAT Solving Algorithms

DPLL Algorithm

You've learned this in class!

- Backtracking search
- Unit propagation
- Pure literal elimination

Modern SAT Solvers (CDCL)

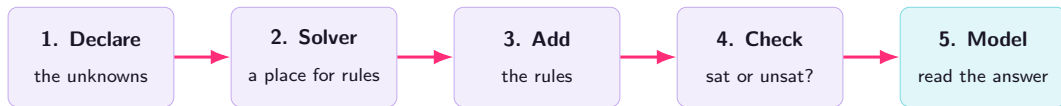
What Z3 actually uses:

- DPLL + learning from conflicts
- When you hit a dead end, analyze *why*
- Add learned clauses to prevent revisiting mistakes
- Vastly faster than vanilla DPLL

Key insight: Z3 is essentially running CDCL in the background. Your constraints get translated into a boolean formula, then solved using highly optimized SAT-solving techniques.

The Z3 Recipe

Five steps. That's the whole API.



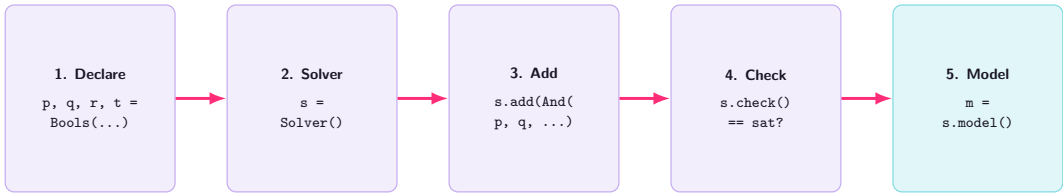
Next up: try the five steps on a quick example.

A quick exercise

Solve this formula:

$$(p \wedge q) \wedge (r \Leftrightarrow p) \wedge (q \Rightarrow t) \wedge (t \vee \neg p)$$

Walk through the five steps:



Propositional Logic

Building blocks

- Variables: $P, Q, R, \dots \in \{\top, \perp\}$
`Bool('P')`

Connectives

- $\neg P$ `Not(P)`
- $P \wedge Q$ `And(P, Q)`
- $P \vee Q$ `Or(P, Q)`
- $P \Rightarrow Q$ `Implies(P, Q)`
- $P \Leftrightarrow Q$ `P == Q`

Satisfiable (sat)

There *exists* an assignment of $\top/\perp$ making every formula true. Z3 hands you one.

Unsatisfiable (unsat)

No assignment works. Your constraints contradict — and Z3 has **proved** it, not merely failed to find one.

Demo 1: Lady or Tiger

An old familiar problem

A prisoner must choose one of **three rooms**.
One holds the **Lady**; the other two hold **tigers**.

- Pick the Lady — he walks free
- Pick wrong — he gets eaten

Every door carries a sign, and:

exactly one sign is true

the other two are lies

Room 1

"The Lady is not in this room"

Room 2

"The Lady is not in this room"

Room 3

"The Lady is in Room 2"

Which door should he open?

Propositional encoding

Exactly one room has the Lady

$$(R_1 \wedge \neg R_2 \wedge \neg R_3) \vee (R_2 \wedge \neg R_1 \wedge \neg R_3) \vee (R_3 \wedge \neg R_1 \wedge \neg R_2)$$

Exactly one sign is true

$$(S_1 \wedge \neg S_2 \wedge \neg S_3) \vee (S_2 \wedge \neg S_1 \wedge \neg S_3) \vee (S_3 \wedge \neg S_1 \wedge \neg S_2)$$

What each sign claims

$$S_1 \Leftrightarrow \neg R_1 \quad S_2 \Leftrightarrow \neg R_2 \quad S_3 \Leftrightarrow R_2$$

The one pattern you'll use over and over

How do you say “exactly one of these is true”?

At least one

$$P_1 \vee P_2 \vee \dots \vee P_n$$

Or(P_1 , P_2 , ..., P_n)

At most one

$$\bigwedge_{i < j} \neg(P_i \wedge P_j)$$

Not(And(P_i , P_j)) for every pair

Exactly one = **at least one** $\wedge$ **at most one** — one queen per row, one digit per cell, one lady behind one door.

Demo 2: Sudoku

The Puzzle

- A 9×9 grid, digits 1–9
- Every row, column and 3×3 box holds each digit exactly once
- `grid[i][j][k]` — “digit $k+1$ sits in cell (i,j) ”
- That is $9 \times 9 \times 9 = \mathbf{729}$ booleans

5	3			7				
6								
							8	
		4						
				9				

Constraint 1 : Only one digit in one cell

No two digits in this cell are the same

$$\neg(g_{ijk} \wedge g_{ijk'}) \quad \text{for all } k \neq k'$$

At least one digit sits in this cell

$$\bigvee_{k=1}^9 g_{ijk}$$

Constraint 2 : No 2 digits in the same row should be the same

P1 : No duplicate digit in a row

$$\neg(g_{ijk} \wedge g_{ij'k}) \quad \text{for all } j \neq j'$$

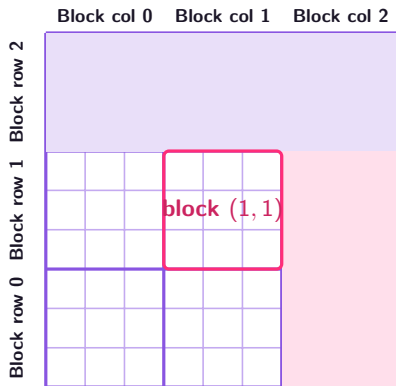
Constraint 3: No two digits in the same column be the same

P1 : No duplicate digit in a column

$$\neg(g_{ijk} \wedge g_{i'jk}) \quad \text{for all } i \neq i'$$

Blocks, block rows and block columns

- A **block** is a 3×3 subgrid.
- The three blocks in the same horizontal strip form a **block row**.
- The three blocks in the same vertical strip form a **block column**.
- We enforce that each digit appears exactly once inside every block.



Constraint 4: Each block doesn't have 2 digits of the same kind

P2 : No duplicate digit in the same block

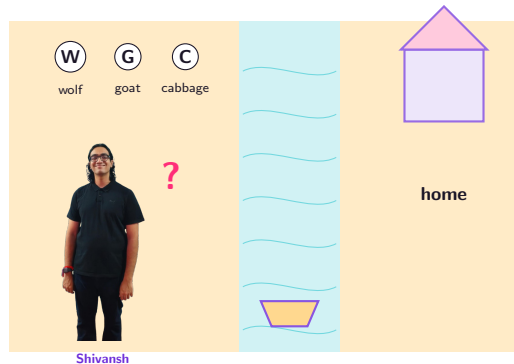
$$\neg(g_{ijk} \wedge g_{i'j'k}) \quad \text{for all } (i,j) \neq (i',j') \text{ in } B_{br,bc}$$

Exercise - 3

Exercise - 3 : Shivansh crosses the river

Farmer **Shivansh** is back from the market with a **Wolf**, a **Goat** and a **Cabbage**. His house is on the **far bank**. He must ferry all three across — and himself.

- The boat holds **Shivansh + at most one** item.
- Only Shivansh rows: he is in the boat *every* trip.
- Leave the **Wolf & Goat** alone $\Rightarrow$ goat eaten.
- Leave the **Goat & Cabbage** alone $\Rightarrow$ cabbage eaten.
- Safety must hold at **every** instant.



Exercise - 3 : the encoding you must produce

Two families of Booleans

- $L_{e,t}$: entity e is on the **left** bank at time t
(right bank = $\neg L$)
- $M_{e,t}$: cargo e rides the boat on step
 $t \rightarrow t+1$

$e \in \{F, W, G, C\}, \quad t = 0 \dots N.$

Five things the formula says

1. start: all on the left at $t=0$
2. goal: all on the right at $t=N$
3. boat: farmer flips banks; ≤ 1 cargo;
boards from farmer's bank, ends across
4. frame: not carried $\Rightarrow$ bank unchanged
5. safety: no unwatched W/G or G/C pair,
ever

Horizon $N = 7$ is satisfiable. Set $N = 6$ and Z3 *proves* no plan exists.

Run the independent checker

From the river_crossing directory: `python3 cross_river.py` then `python3 checker.py cross_river.py`

POP QUIZ !!

The Great Golgappa Challenge

The department is running a golgappa-eating contest: can you finish **30 golgappas** in one sitting? Five TAs entered — but rumour says not everyone has the stomach. Each Boolean is True iff that TA can eat 30.

Encode the clues and return the unique model.

Var	TA		
A	Tamanna	1. A iff E.	(==)
B	Shishir	2. At least one of A, E.	(Or)
C	Shivansh	3. Exactly one of B, C.	(Xor)
D	Anshul	4. If C then not D.	(Implies, Not)
E	Shayna	5. Exactly one of C, D.	(Xor)
		6. C, or not D.	(Or, Not)
		7. Not both B and D.	(Not, And)

NOTE: do not change the variable names.

Fill `get_model()` in `pop/pop.py`, then run `python check.py` from that folder.